

Table des matières

1. Principes généraux
2. Vue d'ensemble des dépendances
3. Phase 1 — Import du Fichier 1 (catalogue produits)
 - 3.1 Pré-résolution des IDs de référence
 - 3.2 Calcul du prix HT
 - 3.3 Séquence de création
4. Phase 2 — Import du Fichier 2 (déclinaisons + stocks)
 - 4.1 Pré-résolution des attributs
 - 4.2 Calcul du delta de prix des déclinaisons
 - 4.3 Séquence de création
5. Phase 3 — Import du Fichier 3 (clients et commandes)
 - 5.1 Déduplication des clients
 - 5.2 Résolution de l'adresse
 - 5.3 Parsing du champ `achat`
 - 5.4 Calcul des totaux de commande
 - 5.5 Séquence de création commande
6. Codes de retour HTTP
7. Structure des messages d'erreur PrestaShop
 - Format XML (défaut)
 - Format JSON (`?output_format=JSON`)
 - Codes d'erreur internes PrestaShop fréquents
8. Gestion des erreurs dans la séquence d'import
9. Points de vigilance spécifiques aux fichiers fournis
10. Ordre d'exécution complet recommandé

Logique_Import.md — Stratégie d'import des fichiers 1, 2 et 3 · PrestaShop 8.x Webservice

1. Principes généraux

- Aucun outil tiers (n8n, Make, ETL, etc.) : uniquement des appels HTTP directs vers le Webservice PrestaShop.
- Authentification : **Basic Auth** avec la clé API comme username et un mot de passe vide.
- Format de requête : corps toujours en **XML** (`Content-Type: application/xml`).

- Format de réponse : XML par défaut, JSON avec `?output_format=JSON`.
- Toutes les requêtes de création utilisent **POST** et retournent **201 Created**.
- Toutes les mises à jour utilisent **PUT** avec le corps complet de la ressource.

2. Vue d'ensemble des dépendances

```

Fichier 1 (produits)
└─ POST /api/products          → id_product[]
   └─ PUT /api/stock_availables → (stock produit simple)

Fichier 2 (déclinaisons + stock)
└─ POST /api/combinations      → id_combination[] (dépend id_product)
   └─ PUT /api/stock_availables → (stock par déclinaison)

Fichier 3 (clients + commandes)
├─ POST /api/customers         → id_customer[]
├─ POST /api/addresses         → id_address[] (dépend id_customer)
├─ POST /api/carts             → id_cart[] (dépend id_customer,
id_address)
├─ PUT /api/carts/{id}         → cart_rows (dépend id_product,
id_combination)
├─ POST /api/orders            → id_order[] (dépend id_cart,
id_address)
└─ POST /api/order_histories   → état initial (dépend id_order)

```

3. Phase 1 — Import du Fichier 1 (catalogue produits)

3.1 Pré-résolution des IDs de référence

Avant de créer les produits, récupérer les IDs des entités existantes :

```

GET /api/categories?filter[name]=Akanjo&display=[id]
GET /api/categories?filter[name]=Accessoire&display=[id]
GET /api/tax_rule_groups?display=full          → repérer les groupes 11,65 % et
5,60 %
GET /api/languages?filter[active]=1&display=[id,iso_code]
GET /api/currencies?filter[active]=1&display=[id,iso_code]

```

3.2 Calcul du prix HT

PrestaShop stocke les prix **hors taxes**. Le fichier 1 fournit le prix TTC :

```
prix_HT = prix_TTC / (1 + taux_taxe / 100)
```

```
T_01 : 12,50 / 1,1165 = 11,2050...
P_01 : 18,99 / 1,1165 = 17,0089...
C_03 : 5,00 / 1,0560 = 4,7348...
M_02 : 56,00 / 1,0560 = 53,0303...
```

3.3 Séquence de création

Pour chaque ligne du fichier 1 :

1. POST /api/products → noter id_product
2. GET /api/stock_availability
?filter[id_product]={id_product}
&filter[id_product_attribute]=0 → noter id_stock_available
3. PUT /api/stock_availability/{id} → quantity = stock_initial
(Fichier 2)

C_03 et M_02 n'ont pas de déclinaison : leur stock est directement dans **stock_available** avec **id_product_attribute = 0**.

4. Phase 2 — Import du Fichier 2 (déclinaisons + stocks)

4.1 Pré-résolution des attributs

```
GET /api/product_options?display=full → repérer "taille", "couleur"
GET /api/product_option_values?display=full → repérer ngoza, kely, mainty, fotsy
```

Si les valeurs n'existent pas, les créer :

```
POST /api/product_options → créer l'attribut "taille"
POST /api/product_option_values → créer la valeur "ngoza" liée à l'attribut
```

4.2 Calcul du delta de prix des déclinaisons

Le champ **price** d'une **combination** est un **delta** par rapport au prix du produit parent :

```
T_01 parent price HT = 11,2050
T_01/ngoza prix TTC = 12,50 → prix HT = 11,2050 → delta = 0,0000
T_01/kely prix TTC = 15,00 → prix HT = 13,4305 → delta = +2,2255

P_01 parent price HT = 17,0089
P_01/mainty prix TTC = 23,49 → prix HT = 21,0400 → delta = +4,0311
P_01/fotsy prix TTC = 18,99 → prix HT = 17,0089 → delta = 0,0000
```

4.3 Séquence de création

Pour chaque ligne du fichier 2 avec une variante non vide :

1. POST /api/combinations → noter id_combination
(id_product + product_option_values + price delta + minimal_quantity)
2. GET /api/stock_availability
?filter[id_product]={id_product}
&filter[id_product_attribute]={id_combination}
3. PUT /api/stock_availability/{id} → quantity = stock_initial

5. Phase 3 — Import du Fichier 3 (clients et commandes)

5.1 Déduplication des clients

Rakoto apparaît deux fois avec le même email. Avant de créer un client, vérifier s'il existe :

```
GET /api/customers?filter[email]={EMAIL}&display=[id]
```

- Si [] → POST /api/customers → noter id_customer
- Sinon → réutiliser l'id_customer existant

5.2 Résolution de l'adresse

```
GET /api/addresses?filter[id_customer]={id_customer}&display=[id,alias,city]
```

Si aucune adresse correspondant à la ville du CSV n'existe :

```
POST /api/addresses → id_address_delivery
```

5.3 Parsing du champ achat

Format brut : [("T_01";3;"ngoza"), ("C_03";1;"")]

Algorithme de parsing :

1. Supprimer les caractères [] " autour
2. Séparer par),(pour isoler chaque ligne
3. Pour chaque ligne : split par ; → [reference, quantité, variante]
4. Résoudre id_product via GET /api/products?filter[reference]={REF}
5. Si variante non vide : résoudre id_combination via

```
GET /api/combinations?filter[id_product]={id}&display=full
→ trouver la combinaison dont product_option_values correspond à la variante
6. Si variante vide : id_product_attribute = 0
```

5.4 Calcul des totaux de commande

```
Pour chaque ligne du panier :
    sous_total_TTC = quantité × prix_TTC_déclinaison
    sous_total_HT  = quantité × prix_HT_déclinaison

total_products_wt = Σ sous_total_TTC
total_products    = Σ sous_total_HT
total_paid        = total_products_wt + total_shipping (0 si non renseigné)
```

5.5 Séquence de création commande

```
1. POST /api/carts           → id_cart
2. PUT  /api/carts/{id_cart} → injecter cart_rows
3. POST /api/orders          → id_order (corps complet avec totaux)
4. POST /api/order_histories → enregistrer l'état initial
```

6. Codes de retour HTTP

Code	Signification	Cas d'usage PrestaShop
200	OK	GET réussi, PUT réussi
201	Created	POST réussi — le corps contient la ressource créée avec son id
204	No Content	DELETE réussi (certaines versions)
304	Not Modified	Ressource inchangée (cache via Content-Sha1)
400	Bad Request	XML malformé, champ requis absent, valeur invalide, contrainte violée
401	Unauthorized	Clé API absente, incorrecte, désactivée, ou WebService désactivé
403	Forbidden	Clé sans permission pour cette ressource ou cette méthode
404	Not Found	Ressource inexistante, URL incorrecte, réécriture d'URL mal configurée
405	Method Not Allowed	Méthode HTTP non supportée pour cette ressource
500	Internal Server Error	Erreur serveur, exception PHP, contrainte base de données

7. Structure des messages d'erreur PrestaShop

Format XML (défaut)

```
<?xml version="1.0" encoding="UTF-8"?>
<prestashop>
  <errors>
    <error>
      <code><![CDATA[80]]></code>
      <message><![CDATA[The product with id 9999 does not exist.]]></message>
    </error>
  </errors>
</prestashop>
```

Une réponse peut contenir plusieurs `<error>` si plusieurs validations échouent simultanément.

Format JSON (`?output_format=JSON`)

```
{
  "errors": [
    {
      "code": 80,
      "message": "The product with id 9999 does not exist."
    }
  ]
}
```

Codes d'erreur internes PrestaShop fréquents

Code interne	Signification
17	Clé Webservice invalide ou permissions insuffisantes
32	Ressource non trouvée
80	Entité liée (FK) introuvable
100	Erreur de validation (champ manquant ou invalide)
200	Erreur SQL / contrainte de base de données

8. Gestion des erreurs dans la séquence d'import

Pour chaque appel HTTP :

- SI code == 201 ou 200 :
 - Extraire l'ID depuis le corps XML/JSON
 - Continuer la séquence
- SI code == 400 :
 - Lire le message d'erreur


```
PUT /api/stock_availability (C_03, M_02 uniquement – pas de déclinaison)
```

Phase 2 : Fichier 2

```
POST /api/combinations x 4 (T_01/ngoza, T_01/kely, P_01/mainty,  
P_01/fotsy)
```

```
PUT /api/stock_availability x 4 (une par déclinaison)
```

Phase 3 : Fichier 3

```
POST /api/customers (Rakoto si absent, Rajao)
```

```
POST /api/addresses (une par client)
```

```
POST /api/carts x 3
```

```
PUT /api/carts/{id} x 3 (cart_rows)
```

```
POST /api/orders x 3
```

```
POST /api/order_histories x 3 (état initial)
```